

Lung Cancer— A Look into Lifestyles

Team Members:

IB 112351008 LIEN PEI HAN
109301042 MI JOU HSIAO
109301022 Chih Yu Chen
STAT 111304041 ShIU JIA YU
MIS 109306045 Huang Yun Chien
COM 112300396 Jerzy Kopinski



Table of contents

01

Introduction

02

Methodology

03

Project Timeline

04

QA



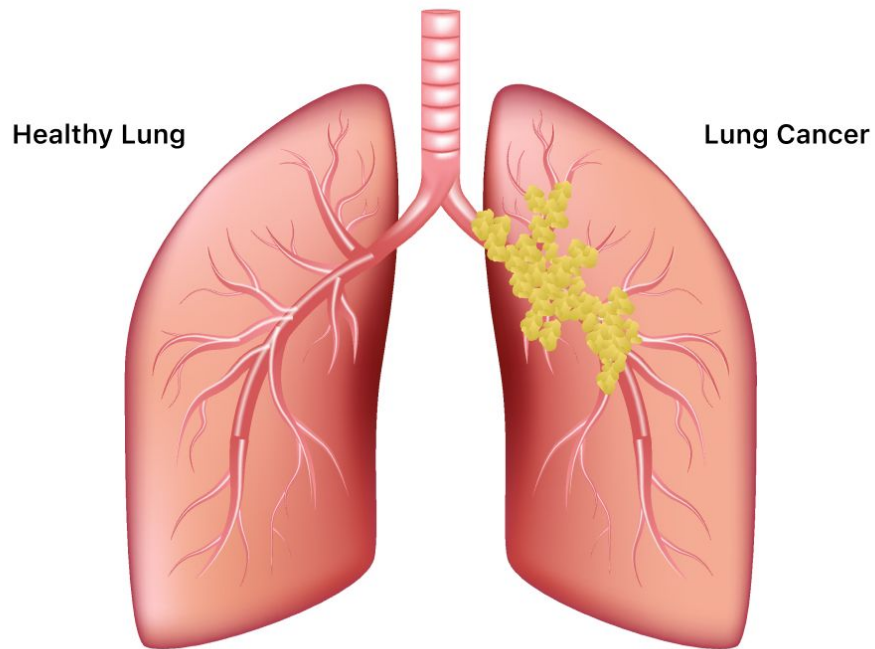
01

Introduction



What is Lung Cancer

- Lung cancer is a type of cancer that starts in the lungs. It is one of the most common cancers worldwide and a leading cause of cancer-related deaths.





What cause Lung cancer?

According to *American Cancer Society*, the primary factors that cause lung cancer Are the following:

Risk factors you can change

- Tobacco smoke
- Secondhand smoke
- Exposure to radon
- Exposure to asbestos
- Taking certain dietary supplements
(beta-carotene supplements)
- Arsenic in drinking water

Risk factors you cannot change

- Air pollution
- Personal or family history of lung cancer



Project Goal: Predicting Lung Cancer

We are curious about among all the lifestyle factors, which of them would have much higher possibilities to cause lung cancer.

02

Methodology



Data Introduction

Data name : Lung cancer patient's lifestyle
-12 columns*10000 rows/continuous data

80/20 train-test split



Data Introduction

ID	Gender	Age	Children	Smoker	Employed	Years Worked	Income Level	Social Media	Online Gaming	Cancer
25137	0	23	3	0	0	0	0	1	0	1
13063	0	55	4	1	0	0	0	0	0	1
61685	1	43	3	0	1	25	0	1	1	1
49303	1	24	0	1	1	15	2	1	1	0
84442	1	54	5	1	0	0	0	0	1	0
99609	1	19	0	0	1	2	0	1	1	0
55657	1	67	0	1	0	0	0	1	0	1
40949	0	19	2	0	1	34	0	0	1	0
11655	0	41	1	0	0	0	0	0	1	0

input

output

- ! Smoker, Employed, Social media, Online gaming: 0=no 1=yes
- ! Gender: 0=male 1=female
- ! Income level: 0=low 1=medium 2=high

Data Preprocessing

Step 1

```
data.isna().sum()
```

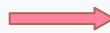
```
ID          0
Gender       0
Age          0
Children     0
Smoker       0
Employed     0
Years Worked 0
Income Level 0
Social Media 0
Online Gaming 0
Cancer       0
dtype: int64
```

no NA data!

Step 2

Delete duplicate data

10000 rows



8281 rows

Step 3

```
Data columns (total 12 columns):
#  Column      Non-Null Count  Dtype
---  -
0   ID          10000 non-null    int64
1   Gender       10000 non-null    int64
2   Age          10000 non-null    int64
3   Children     10000 non-null    int64
4   Smoker       10000 non-null    int64
5   Employed     10000 non-null    int64
6   Years Worked 10000 non-null    int64
7   Income Level 10000 non-null    int64
8   Social Media 10000 non-null    int64
9   Online Gaming 10000 non-null    int64
10  Cancer       10000 non-null    int64
11  dtypes: int64(12)
```

all integer type

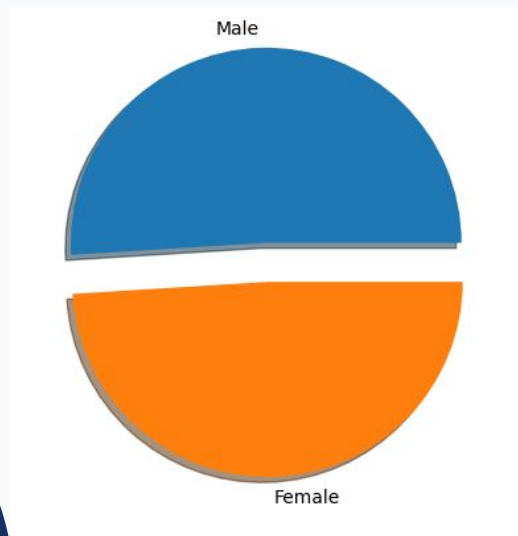
Step 4

Standardization to
Numeric data:
Age/Years worked/Children

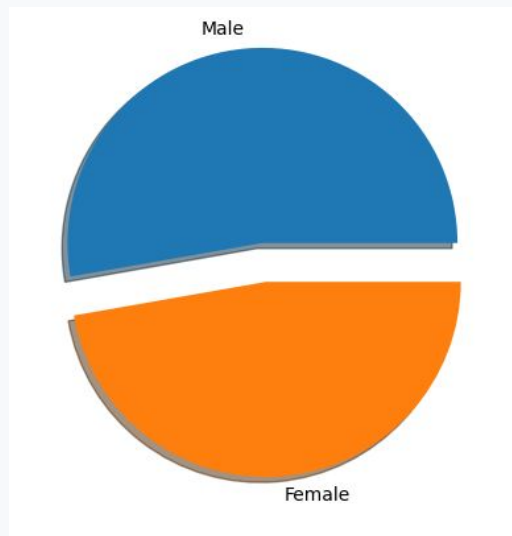
EDA Results

Male
Female

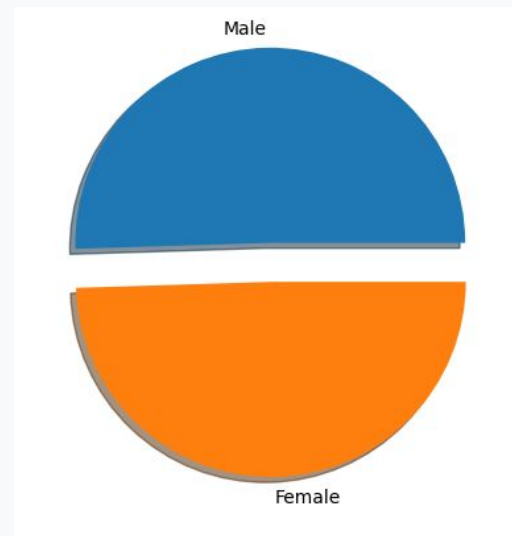
Gender Proportion
in all data



Healthy Gender
Proportion



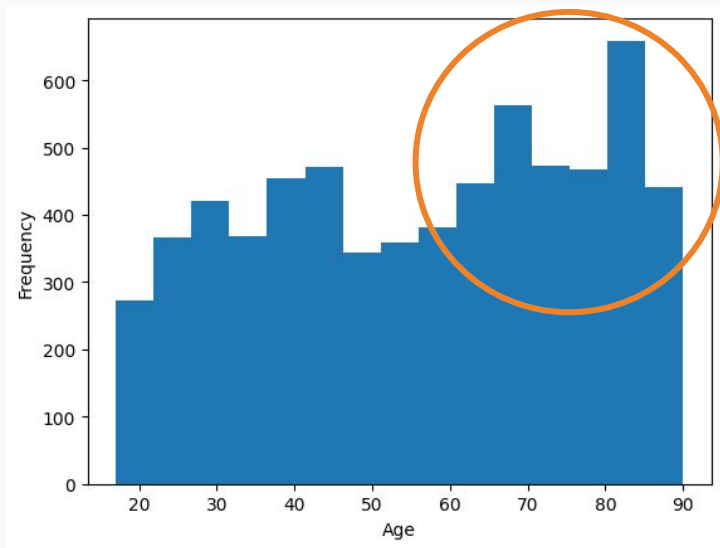
Gender Proportion
with cancer



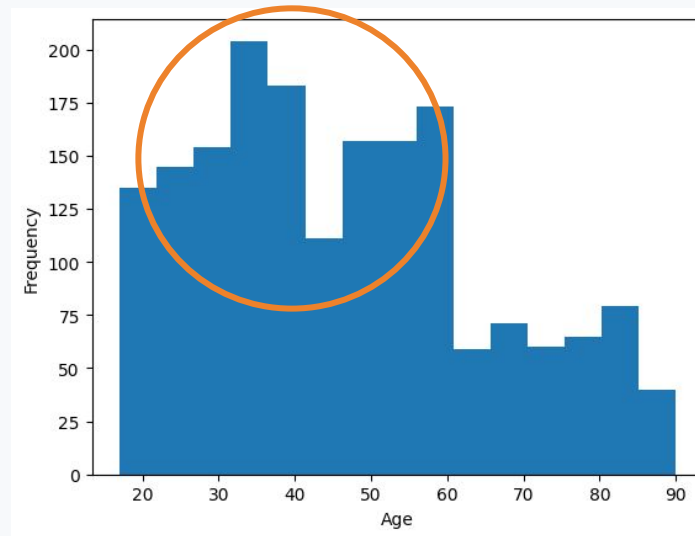
Gender is not a evident factor of lung cancer due to the close proportion.

EDA Results

Proportion of Frequency and Age with cancer



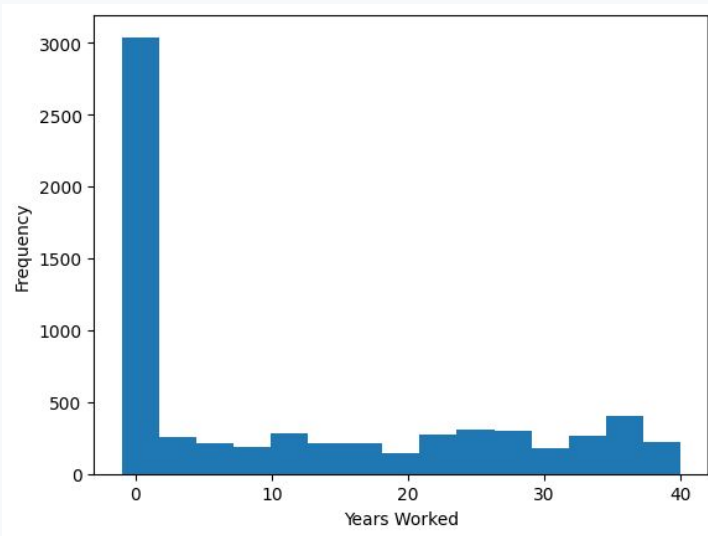
Proportion of Frequency and Age without cancer



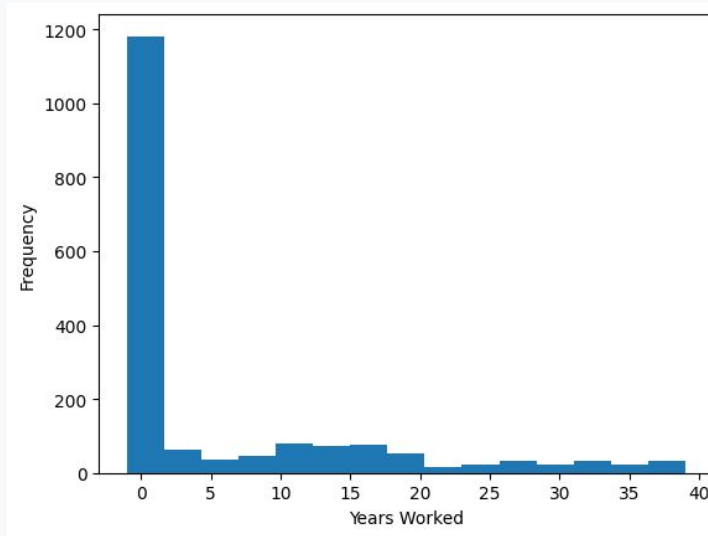
Age might be one of the factors affecting the proportion of cancer

EDA Results

Proportion of Frequency and Years worked with cancer



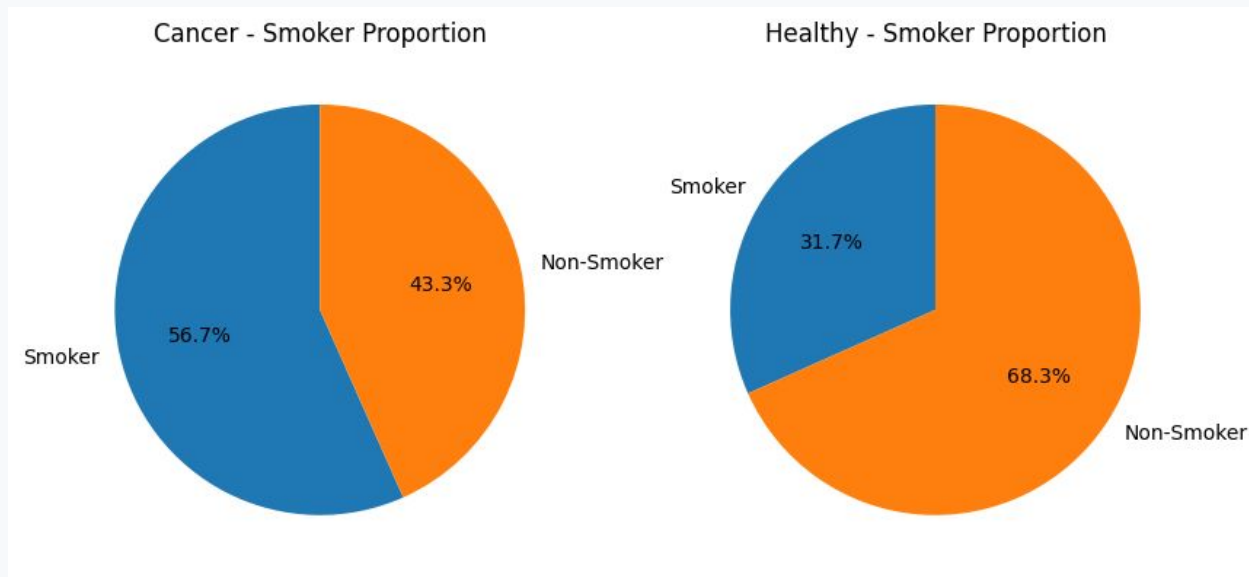
Proportion of Frequency and Years worked without cancer



No relevance between the proportion of cancer and years worked

EDA Results

Smoker Proportion between cancer and healthy



Expectedly, smoking is one of the most important factors increasing the probability of cancer.

Methods

Supervised Learning

01



Machine Learning

- SVM
- K-Nearest Neighbors (KNN)

02



Deep Learning

- Artificial Neural Network (ANN)



03

Analysis Result





Preparation for SVM Model

All factors besides ['Cancer'] as Input and ['Cancer'] as Output

```
1 # input
2 X = data.drop(columns = ['Cancer'])
3 # output
4 y = data['Cancer']
```

Splitting data into test size=0.2 and train size= 0.8

```
1 from sklearn.model_selection import train_test_split
2 X_train, X_test, y_train, y_test = train_test_split(X, y, test_size=0.2, random_state=100)
```

Utilized Best Parameter to Fit the Model

GridSearchCV :

Using cross-validation to search for the best parameter combination.

```
12 # 調参
13 param_grid = {
14     'C': [2400, 2500, 3000],
15     'gamma': [ 2.4, 2.5, 2.6]
16 }
17
18 grid_search = GridSearchCV(SVC(class_weight='balanced'), param_grid, cv=5)
19 grid_search.fit(X_train, y_train)
20
21 print(f'Best Model Parameters: {grid_search.best_params_}')
```

Fits the model :

Using the best parameters for the training data.

```
Best Model Parameters: {'C': 2400, 'gamma': 2.5}
```



Model Evaluation

Train SVM with best parameters

```
1 svm = SVC(**grid_search.best_params_)
2 svm.fit(X_train, y_train)
3 y_pred_best = svm.predict(X_test)
```

Training result: best model reaches the accuracy of 0.9957

Best Model Accuracy:0.9957

Best Model precision:0.9982

Best Model recall:0.9964

Best Model f1:0.9973

Confusion Matrix:

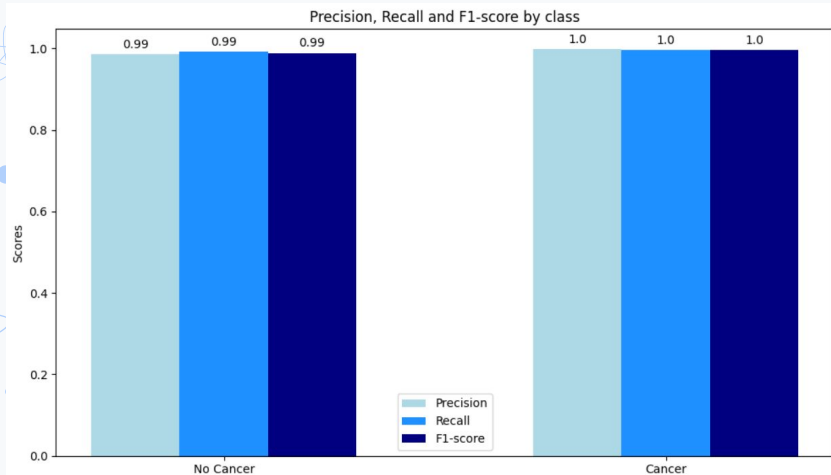
```
[[ 282   2]
 [   4 1119]]
```

*Evaluation process

```
5 # 模型評估
6 accuracy = accuracy_score(y_test, y_pred_best)
7 precision = precision_score(y_test, y_pred_best)
8 recall = recall_score(y_test, y_pred_best)
9 f1 = f1_score(y_test, y_pred_best)
10 conf_matrix = confusion_matrix(y_test, y_pred_best)
```

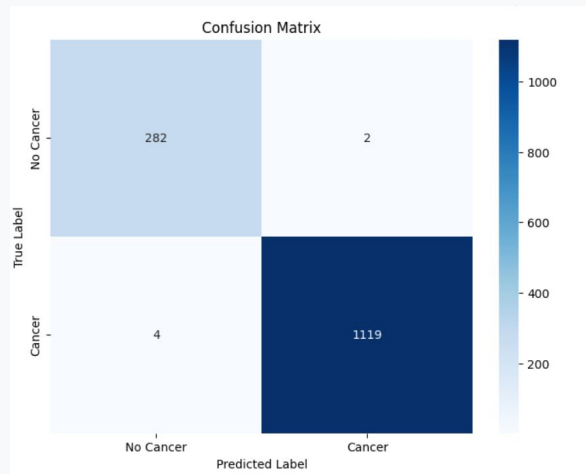
Visualization of the result

Model Accuracy: 0.9957



Confusion Matrix:

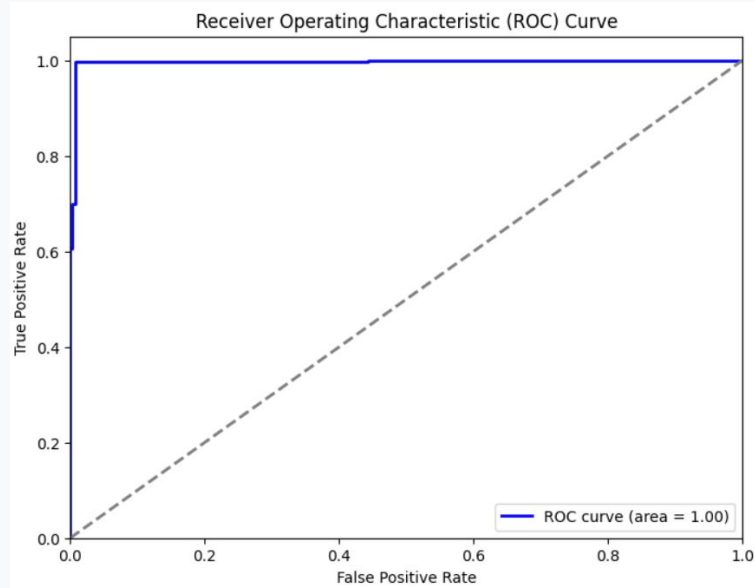
- True positive rate: 0.9964
- False positive rate: 0.007



Visualization of the result

ROC Curve

- AUC = 1.00 in 100% of cases, the model correctly distinguishes between positive and negative cases.





Preparation for K-Nearest Neighbors

Packages used for KNN

```
import numpy as np
import pandas as pd
from sklearn.model_selection import train_test_split
from sklearn.preprocessing import StandardScaler
from sklearn.neighbors import KNeighborsClassifier
from sklearn.metrics import accuracy_score, classification_report, confusion_matrix
```

Splitting data into test size=0.2 and train size= 0.8

```
[25] # data split
      X_train, X_test, y_train, y_test = train_test_split(X, y, test_size=0.2, random_state=42)
```



Training Process for KNN

Using StandardScaler() to transform continuous features

```
[26] #data normalization
      continuous_features = ['Age', 'Years Worked']
      discrete_features = ['Gender', 'Children', 'Smoker', 'Employed', 'Income Level', 'Social Media', 'Online Gaming']

      scaler = StandardScaler()
      X_train_continuous = scaler.fit_transform(X_train[continuous_features])
      X_test_continuous = scaler.transform(X_test[continuous_features])

      X_train_scaled = pd.DataFrame(X_train_continuous, columns=continuous_features)
      X_test_scaled = pd.DataFrame(X_test_continuous, columns=continuous_features)

      X_train_scaled[discrete_features] = X_train[discrete_features].reset_index(drop=True)
      X_test_scaled[discrete_features] = X_test[discrete_features].reset_index(drop=True)

      X_train_scaled.head(), X_test_scaled.head()
```

Fits the model with partially transformed features

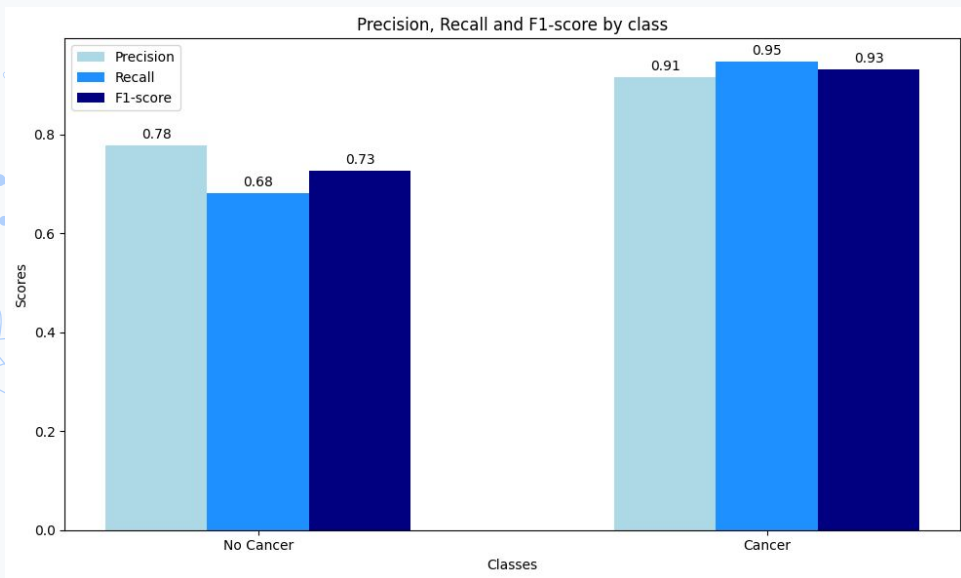
```
[27] knn = KNeighborsClassifier(n_neighbors=5)
      knn.fit(X_train_scaled, y_train)
```

```
↔ KNeighborsClassifier
   KNeighborsClassifier()
```

```
[28] y_pred = knn.predict(X_test_scaled)
```

Visualization of the result

Model Accuracy: 0.8884



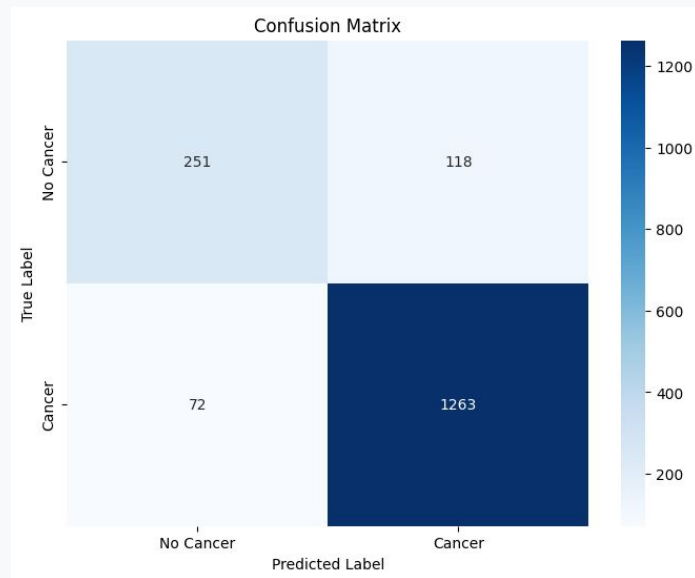
Confusion Matrix:

TP: 1263

TF: 251

FP: 106

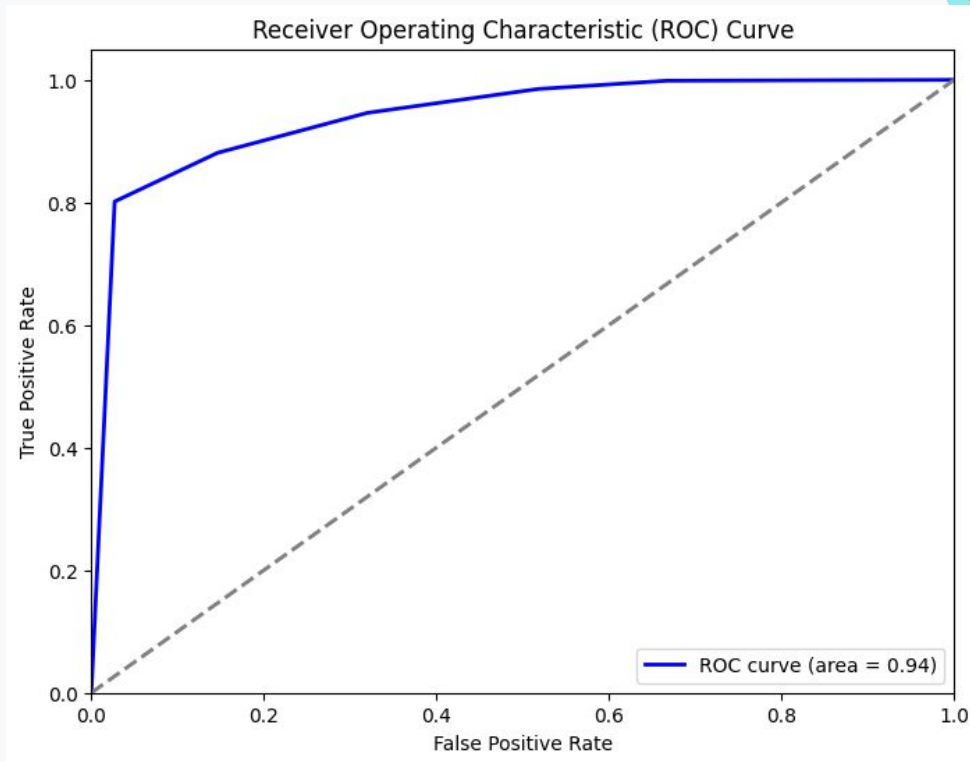
FN: 78



Visualization of the result

ROC Curve

- AUC = 0.94 in 94% of cases, the model correctly distinguishes between positive and negative cases.



Preparation for ANN Model



Initializing SMOTE to fit and resample data

```
from imblearn.over_sampling import SMOTE
import pandas as pd
smote = SMOTE()
X_resampled, y_resampled = smote.fit_resample(X, y)
X_train = X_resampled
y_train = y_resampled
```

```
Cancer
1      7850
0      7850
Name: count, dtype: int64
15700
2000
```

Training Process for ANN

```
# Announce a model  
mymodel = Sequential()  
# i called it "mymodel"
```

```
# Starting to add layer  
mymodel.add(Flatten(input_shape=(9,)))  
# flatten the matrix image into long vector
```

```
# Add 1st hidden layer  
mymodel.add(Dense(28, activation='relu'))
```

```
# Add 2nd hidden layer  
mymodel.add(Dense(14, activation='relu'))
```

```
mymodel.add(Dense(1, activation='sigmoid')) #只有0或1  
# softmax for probability
```

```
mymodel.compile(loss='binary_crossentropy', optimizer='adam', metrics=['accuracy'])
```

There are 9 features for the input

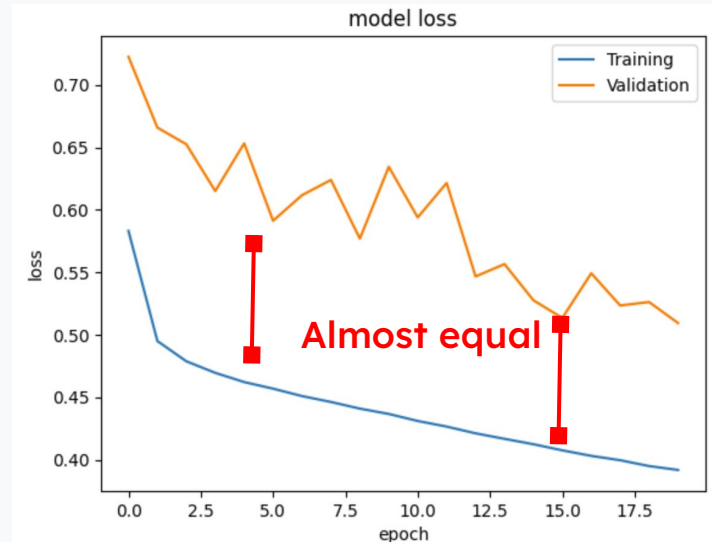
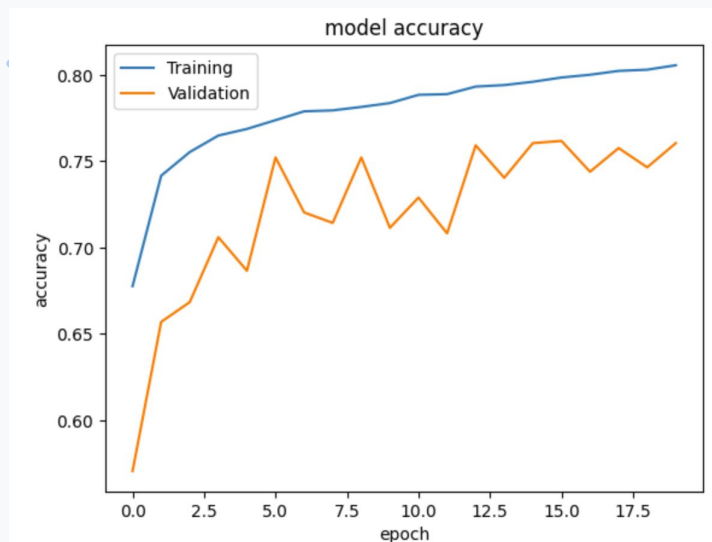
For binary classification tasks: Cancer or not



Training Result BEFORE Dropout

Model Accuracy: the accuracy of both training and validation datasets steadily increases

Model Loss: the loss on both training and validation datasets decreases the epochs



Training Process for ANN(Dropout)

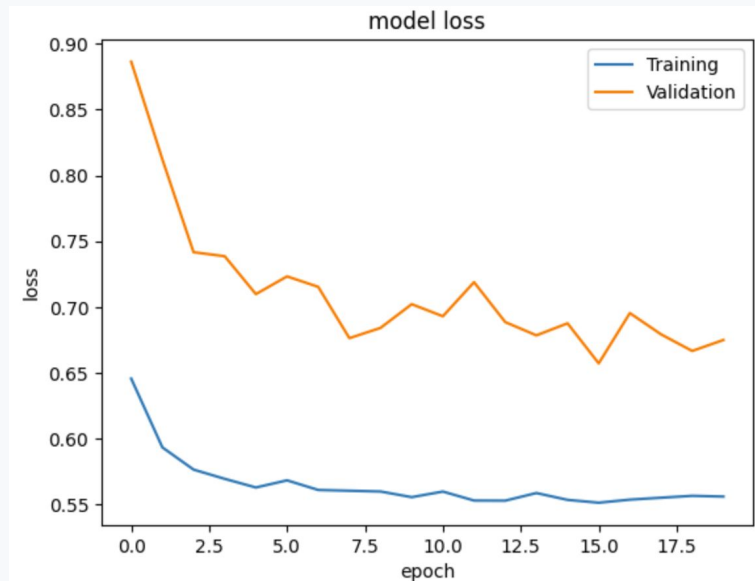
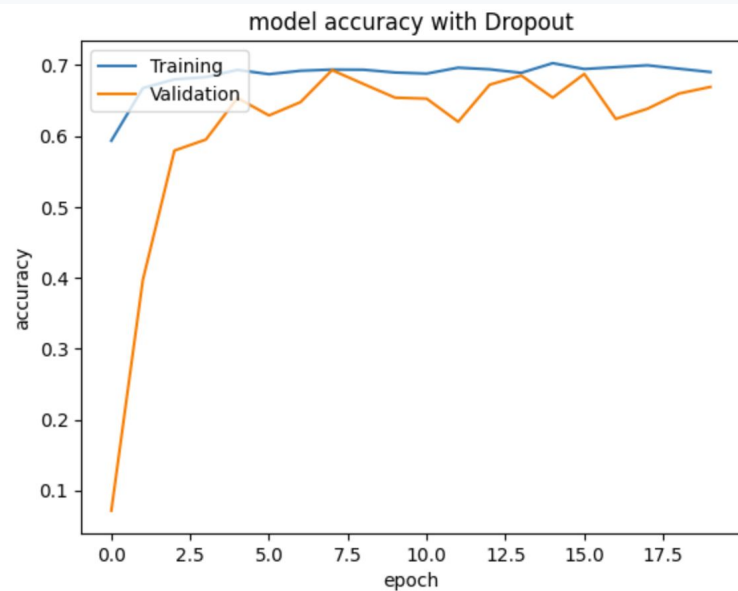
```
# import package for dropout
from keras.layers import Dropout

# Similar step
mymodel_improved = Sequential()
mymodel_improved.add(Flatten(input_shape = (9,)))
#Adding a dropout layer to avoid the overfitting
mymodel_improved.add(Dropout(0.3))
mymodel_improved.add(Dense(28, activation='relu'))
mymodel_improved.add(Dense(14, activation='relu'))
#Adding a dropout layer to avoid the overfitting
mymodel_improved.add(Dropout(0.3))
mymodel_improved.add(Dense(1, activation='sigmoid'))
mymodel_improved.compile(loss='binary_crossentropy', optimizer='adam', metrics=['accuracy'])
```

Training Result When Dropout=0.3

Model Accuracy: the accuracy of both training and validation datasets steadily increases

Model Loss: the loss on both training and validation datasets decreases the epochs



Result Comparison: Before and After Dropout

- In both models, the accuracy of train and test has little difference, so there's no overfitting problem, and it also explains why applying dropout did not provide a significant benefit

Accuracy
81.09%

Before Dropout

Accuracy
79.60%

Dropout = 0.3

Utilized ROC to Find New Threshold

Calculate ROC curve to find optimal threshold: 0.6654

```
#Define predict as the accuracy of the model  
fpr, tpr, thresholds = roc_curve(y_test, predict)
```

```
#choose optimal value, find the balance between trp and fqe  
optimal_idx = np.argmax(tpr - fpr)  
optimal_threshold = thresholds[optimal_idx]
```

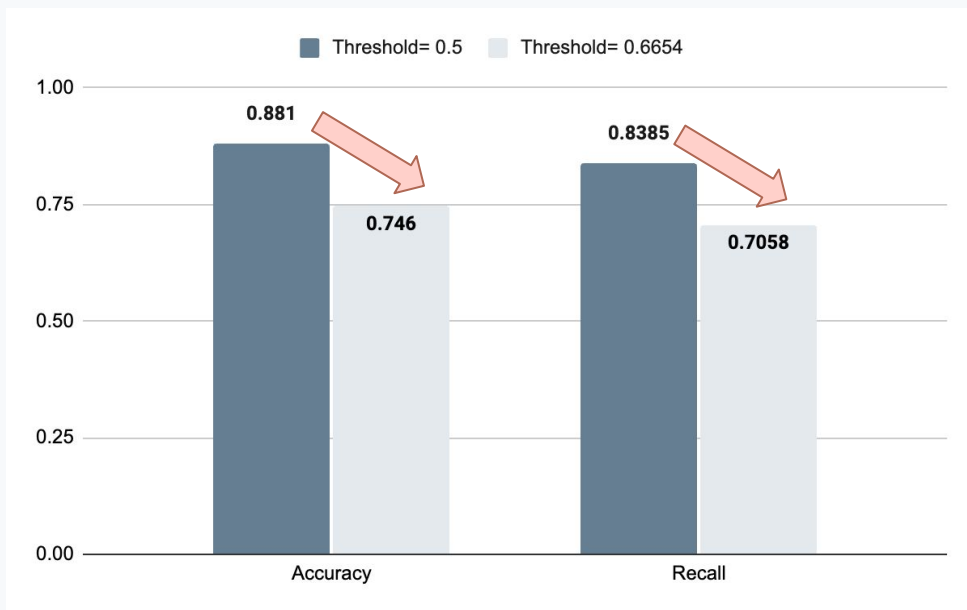
```
optimal_threshold
```

```
0.6654142
```



Result Comparison: Before and After Threshold Adjustment

Both Accuracy and Recall become lower. The original threshold (0.5) is better.



Utilized Activation for Better Model Performance

Softmax activation and more epochs have better performance, which accuracy reaches 0.9133 and recall becomes 0.8994

```
model = Sequential()
model.add(Dense(64, input_dim=X_train.shape[1], activation='relu'))
model.add(Dense(32, activation='relu'))
model.add(Dense(y.shape[1], activation='softmax'))

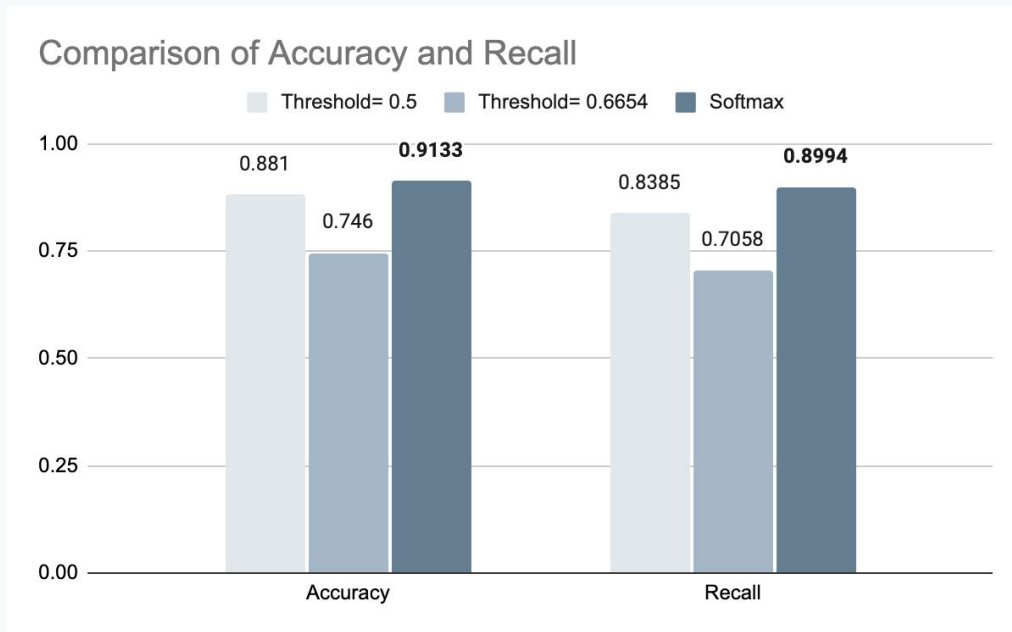
model.compile(optimizer='adam', loss='categorical_crossentropy', metrics=['accuracy'])
es = EarlyStopping(monitor='val_loss', mode='min', verbose=1, patience=20)

model.fit(X_train, y_train, validation_split=0.1, epochs=100, batch_size=5, callbacks = [es], verbose=1)

y_pred = model.predict(X_test)
y_pred_classes = y_pred.argmax(axis=1)
# y_test_classes = y_test.values.argmax(axis=1)
```

Result Comparison: Before and After Threshold Adjustment

Among 3 of the adjustments, Softmax activation has the best predicting performance



04

Conclusion



Interesting Insights from SVM

Features that have significant impact on getting lung cancer:

- Smoker
- Social Media
- Online Gaming

Lifestyle and habits may be main roots for lung cancer

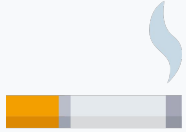
Features that have less relevance impact on getting lung cancer:

- Gender

Gender may be irrelevant in getting lung cancer

	Feature	Importance
0	Gender	-0.000429
1	Age	0.032523
2	Children	0.021078
3	Smoker	0.065451
4	Employed	0.030281
5	Years Worked	0.022651
6	Social Media	0.036218
7	Online Gaming	0.036576
8	Marital_0	0.008798
9	Marital_1	0.000024
10	Marital_2	0.000310
11	Marital_3	-0.000930
12	Income_0	0.003267
13	Income_1	0.016476
14	Income_2	0.009585

Interesting Insights from SVM



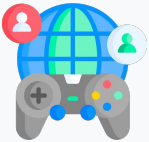
Smoking

- Approximately 80% of lung cancer deaths are due to smoking. Smokers are 15 to 30 times more likely to develop lung cancer compared to non-smokers.



Social Media

- Promoting smoking and vaping habits
- Fragile states lead to unhealthy coping mechanisms like smoking



Online Gaming

- Sedentary lifestyle contributes to health issues that increase cancer risk

Work distribution



Presentation	Lina, Mi Jou Hsiao
Data processing	Chin Yu Chen
EDA, Model training	Shiu Jia Yu, Huang Yun Chien, Jerzy
Lung cancer research	Lien Pei Han
Details and design	Lien Pei Han



Q & A

Thank You